Programming Project 5 Report

Matthew Wilkinson
100434077

Problem Statement:

The goal of the assignment was to extend a basic ray tracing program in C++ and OpenGL by adding three of the four available features. For this project, I implemented the first 3 features on the assignment page. The first feature is creating the Turner Whitted's sphere animation scene by creating a yellow sphere rotating around the scene on a yellow and red checkerboard with a blue background. The second feature was adding new geometric objects, which in my case was adding 2 boxes. The third feature was adding reflection rays so that objects can reflect other objects and the checkerboard floor.

Design:

I started with one of the sample ray tracing programs from the class website and added features over time. The original program already supported rendering spheres and basic Phong Shading, which helped. My design focused on modifying this program and adding the features I selected.

For feature 1, I used the OpenGL timer callback to repeatedly update the sphere and redraw it as it animates throughout the scene. I added code to generate the checkerboard by checking the x and z coordinates where the rays intersect the floor plane.

For feature 2, I had AI help me create the Box3D class since I've already gotten good practice in previous projects. This stores info for each Box3D that will be in the scene. It also includes a method to calculate ray intersections using slab intersection math. I then added the boxes to the scene at coordinates that I saw fit.

For feature 3, I added several helper functions for reflection rays. These functions calculate dot products, reflection directions, and tracing reflected rays throughout the scene. If a reflected ray hits another object/floor, the reflected color is added to the original object color.

Implementation:

The ray tracing program loops through every pixel on the screen and creates a ray from the camera into the scene. The closest object intersection is found, and the color is calculated using Phong Shading.

For feature 1, I modified the program so only the yellow sphere would appear and modified the animation to match the Turner Whitted demo. I also implemented the checkerboard by alternating red and yellow based on the floor intersection point.

For feature 2, I added two cube objects to the scene. The cubes use axis aligned box intersection calculations. These cubes use the same shading and lighting calculations as the sphere and checkerboard in the scene. They also cast shadows.

For feature 3, reflection rays are cast from the surface of an object after the original shading calculations happen. The reflected ray checks for intersections with other objects (spheres/boxes) or the floor (checkerboard). If it hits an object, that part of the color is then blended with the original shaded color to create a mirror like effect.

AI Prompts:

Prompt 1: Why am I getting compile errors?
Prompt 2: Why is my OpenGL window showing a blank screen?
Prompt 3: Create a Box3D class for this project.
Prompt 4: Why are my shadows appearing in wrong locations?
Prompt 5: Why is every object now under a shadow?
Prompt 6: Why is moving the camera making the scene super weird?
Prompt 7: Why is my program stuttering?
Prompt 8: Why is my program crashing?
Prompt 9: Why did my checkerboard disappear?

Testing:

The program was tested by compiling and running the code several times. I checked that the OpenGL window opened correctly and that the objects appeared in the scene. I confirmed that each object has the correct color, has shadows and reflections. The sphere is animating throughout the scene, similar to the demo. The program appears to work as intended.

Figure 1: First Image of Scene

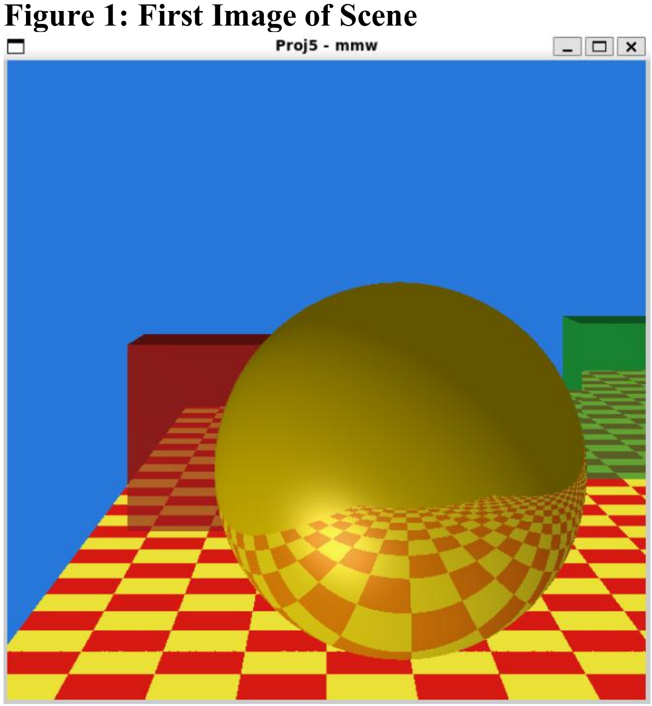


Figure 2: Second Image of Scene (ZOOMED IN a little)

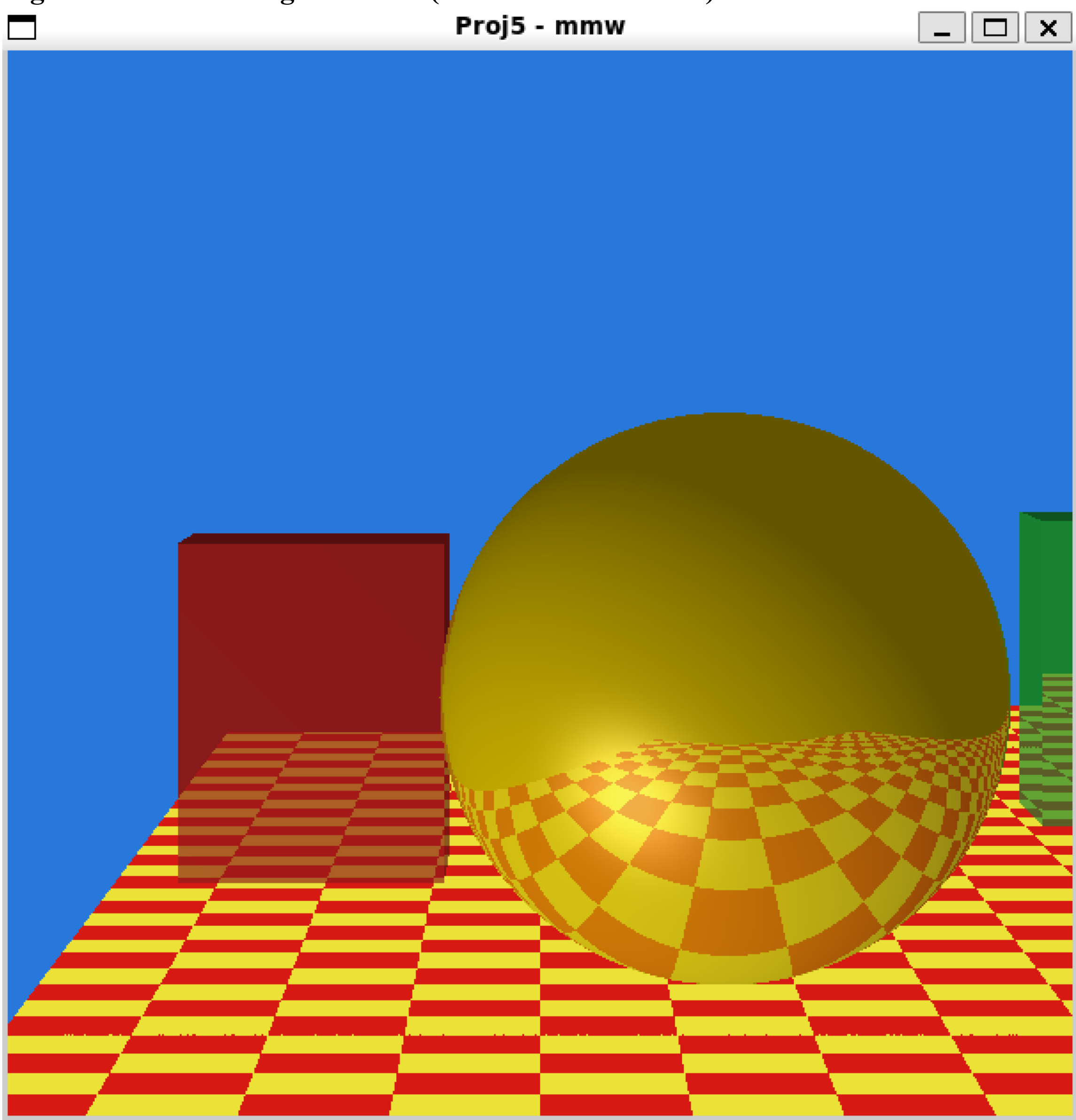


Figure 3: Third Image of Scene (Original View – Showing Shadows)

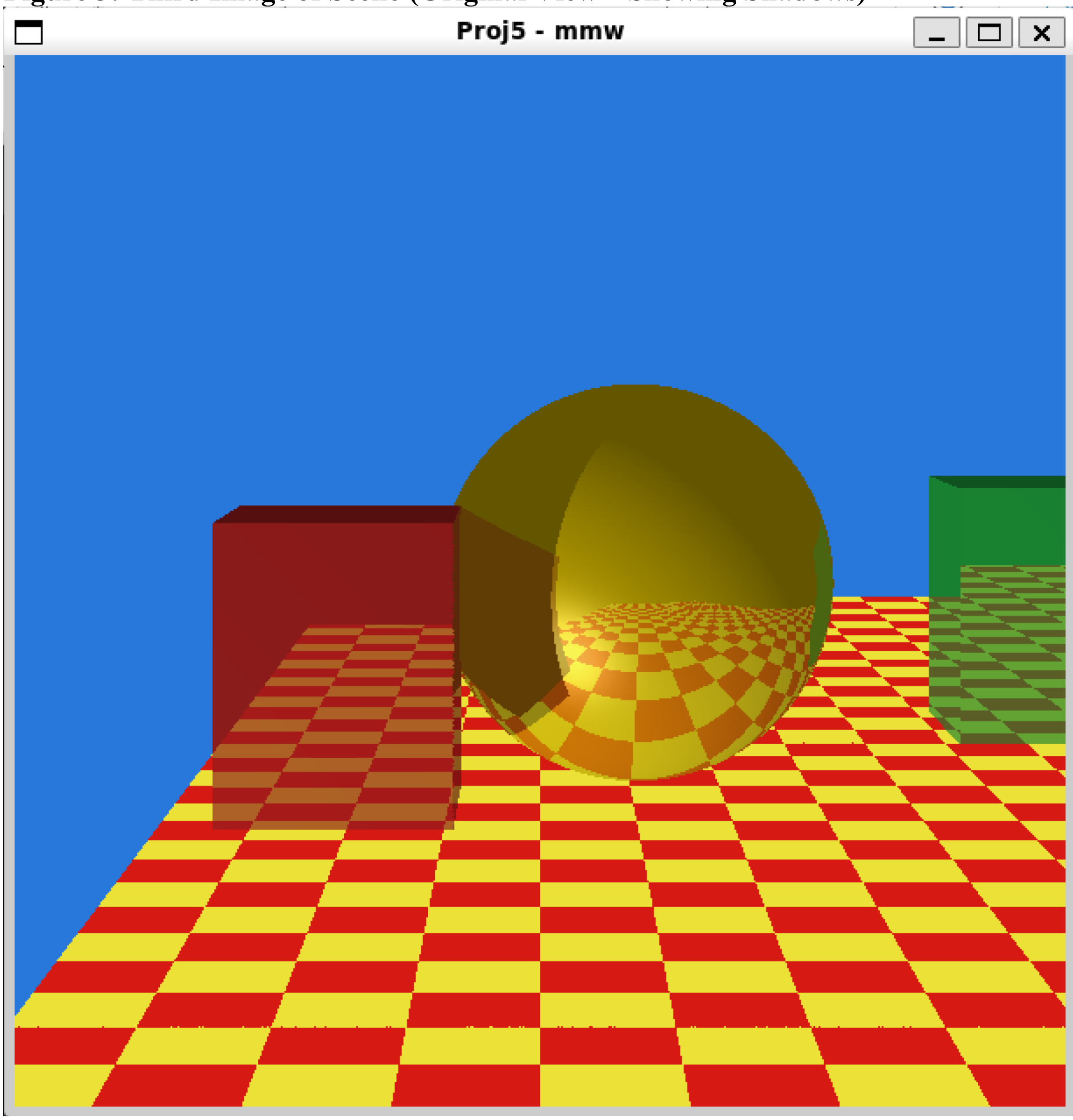
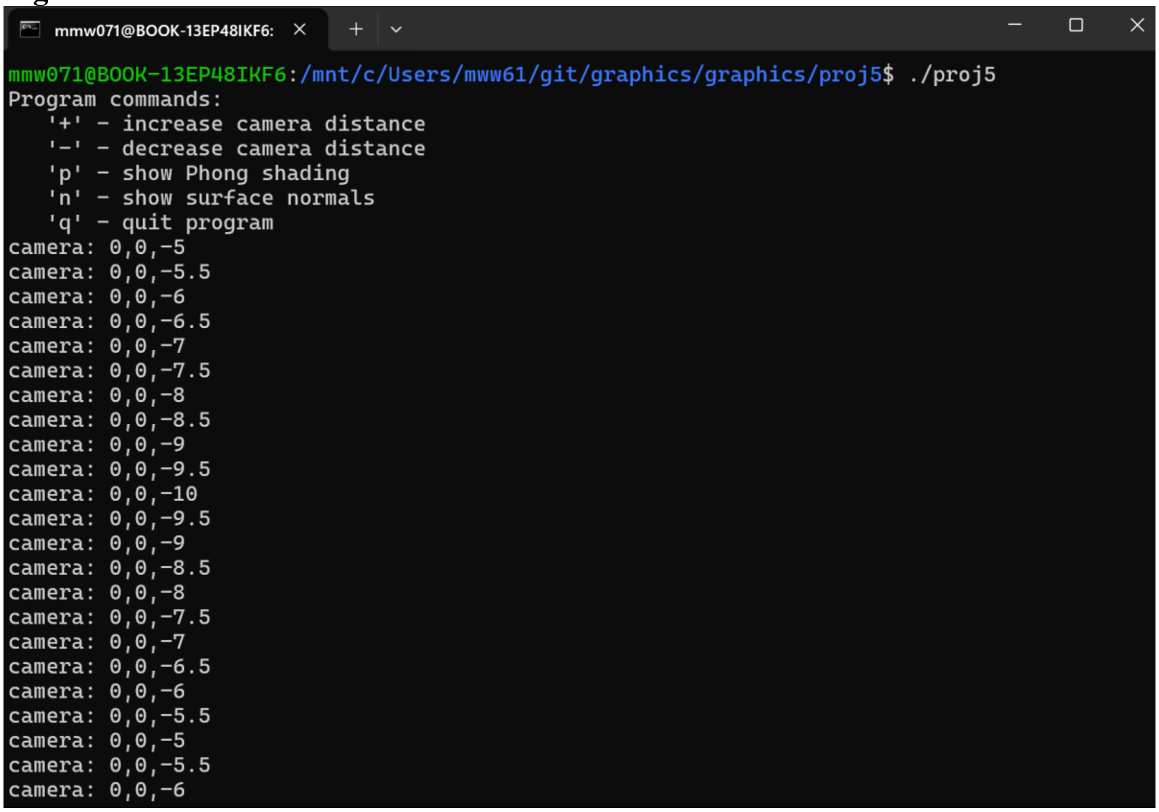


Figure 4: Terminal Screenshot



Conclusions:

I believe the result of the assignment was successful, because it produced an OpenGL window that visually appeared correct. The moving sphere and checkerboard are similar to Turner Whitted's demo. The other geometric objects (boxes in my case) were added to the scene. The reflection rays also improve realism by allowing surfaces to reflect other objects and the floor.

This project is more difficult than previous projects because ray tracing requires more understanding of math and debugging, which there was plenty of. However, it was a nice little project. If I had more time, I would try to add the transmission rays for glass effects, which I think would look pretty cool with all the objects in the scene.